Bash Scripting Exercises

S1 — Hello World + Shebang

Write a bash script called `hello.sh` that:

- Has the correct shebang line
- Prints `Hello, World!` to the screen
- Is executable and can be run as `./hello.sh`

Also answer: why do we use `#!/usr/bin/env bash` instead of `#!/bin/bash`?

> 📄 `03_Bash_Scripting.md` → 3.1 Shebang, Script Structure, Execution

S2 — Variables & Quoting

Write a script that:

- Stores your name in a variable
- Stores a filename with a space in it: `my file.txt`
- Prints: `Hello, I am Alice and my file is: my file.txt`
- Demonstrates what goes wrong if you don't quote the variable containing the space

> 📄 `03_Bash_Scripting.md` → 3.2 Variables, Quoting Rules, export, env

S3 — User Input & Conditionals

Write a script that:

- Asks the user: `Enter a number:`
- Reads the input
- If the number is greater than 10: prints `Big number!`
- If equal to 10: prints `Exactly 10!`
- Otherwise: prints `Small number!`

> 📄 `03_Bash_Scripting.md` → 3.3 Input/Output + 3.4 Conditionals

S4 — For Loop & Arrays

Write a script that:

- Stores a list of 5 server names in an array: `web01 web02 web03 db01 db02`
- Loops through the array
- For each server prints: `Checking server: web01` etc.
- After the loop prints the total count: `Total servers: 5`

> 📄 `03_Bash_Scripting.md` → 3.5 Loops + 3.7 Arrays

S5 — Functions & Exit Codes

Write a script that:

- Has a function called `check_file` that takes one argument (a file path)
- If the file exists: prints `File found: /path/to/file` and returns exit code `0`
- If the file does not exist: prints `File missing: /path/to/file` to stderr and returns exit code `1`
- Calls the function with two paths — one that exists (`/etc/passwd`) and one that doesn't (`/tmp/fakefile.txt`)
- After each call checks `$?` and prints `Check passed` or `Check failed`

> 📄 `03_Bash_Scripting.md` → 3.6 Functions + 3.9 Exit Codes

S6 — While Loop & File Processing

Write a script that:

- Reads `/etc/passwd` line by line using a `while` loop
- For each line extracts the username (field 1) and shell (field 7) using `cut`
- Prints: `User: root | Shell: /bin/bash`
- Only prints users whose shell is `/bin/bash` or `/bin/sh` (skip all others)
- At the end prints: `Total bash/sh users: N`

Also answer: why is `while IFS= read -r line` the correct pattern and what does each part do?

> 📄 `03_Bash_Scripting.md` → 3.5 Loops — for, while, until, break, continue + 3.3 Input/Output

---

S7 — String Manipulation

Write a script that:

- Takes a full file path as an argument: e.g. `/var/log/nginx/access.log.gz`
- Using **only parameter expansion** (no `basename`, no `dirname`, no `sed`):
  - Prints the filename: `access.log.gz`
  - Prints the directory: `/var/log/nginx`
  - Prints the extension: `gz`
  - Prints the filename without extension: `access.log`
- If no argument is provided, prints usage to stderr and exits with code `1`

> 📄 `03_Bash_Scripting.md` → 3.8 String Manipulation — Parameter Expansion

S8 — Error Handling & `trap`

Write a script that:

- Enables strict mode at the top: `set -euo pipefail`

- Creates a temporary directory using `mktemp -d`
- Sets a `trap` to automatically delete the temp directory on any exit — success, error, or signal
- Creates a file inside the temp directory
- Intentionally tries to `cat` a file that doesn't exist (to trigger the error)
- The trap should fire and clean up — prove it by printing `Cleanup done` inside the trap

Also answer: what is the difference between trapping `EXIT` vs trapping `ERR`?

> 📄 `03_Bash_Scripting.md` → 3.9 Exit Codes & Error Handling — set -e, set -o pipefail, trap

---

S9 — Command Substitution & Process Substitution

Write a script that:

- Stores today's date in a variable using `$()`
- Counts the number of running processes using `$()`
- Uses process substitution `<()` to compare the sorted contents of `/etc/hosts` and `/etc/hostname` using `diff` — without creating any temp files
- Prints: `Today is: 2026-03-10 | Running processes: 142`

Also answer: what is the key difference between `$()` and backticks, and what is the pipe subshell problem that process substitution solves?

> 📄 `03_Bash_Scripting.md` → 3.10 Process & Command Substitution — $(), <(), >()

S10 — Associative Arrays & Case Statement

Write a script that:

- Declares an associative array mapping environment names to database hosts:
  - `production` → `prod-db.internal`
  - `staging` → `staging-db.internal`
  - `dev` → `localhost`
- Takes one argument (environment name)
- Uses a `case` statement to validate the argument is one of the three valid environments — exits with error if not
- Looks up the database host from the associative array
- Prints: `Connecting to production database at: prod-db.internal`
- If no argument given, defaults to `dev`

> 📄 `03_Bash_Scripting.md` → 3.7 Arrays & Associative Arrays + 3.4 Conditionals

S11 — Script Hardening & Validation

Write a production-grade script called `deploy.sh` that:

- Has full strict mode at the top (`set -euo pipefail` + IFS)
- Accepts exactly two arguments: `environment` and `version`

- Validates environment must be one of: `production`, `staging`, `dev` — exits with usage message if not
- Validates version matches the pattern `vX.Y.Z` (e.g. `v1.2.3`) using regex — exits with error if not
- Checks that `curl` and `docker` are installed before proceeding — exits with clear error if either is missing
- If environment is `production`, prompts: `Deploying to PRODUCTION. Type yes to confirm:` and aborts if not confirmed
- Has a `readonly` constant `DEPLOY_LOG=/var/log/deploy.log`
- Logs every action with a timestamp to both screen and log file using `tee`

> 📄 `03_Bash_Scripting.md` → 3.15 Script Hardening — set -euo pipefail, Defensive Patterns

---

S12 — Parallel Execution & Job Control

Write a script that:

- Has an array of 6 servers: `web01 web02 web03 db01 db02 cache01`
- SSHs into each server in parallel and runs `uptime`
- Collects all PIDs into an array
- Waits for all jobs to finish
- Checks each job's exit code individually — if any SSH failed, prints `FAILED: web01` to stderr
- At the end prints either `All servers OK` or `X servers failed`
- Limits output so servers don't print interleaved — redirect each to a separate temp file and print after

> 📄 `03_Bash_Scripting.md` → 3.12 xargs & Parallel Execution + 3.6 Functions

S13 — Regex & Input Validation

Write a script that:

- Reads a CSV file line by line (skip the header)
- Each line has format: `name,email,ip_address`
- Validates each field using `[[ =~ ]]`:
  - Email must match a basic pattern: `something@something.something`
  - IP must be a valid IPv4: four octets 0–255
- If validation passes: prints `✓ Valid: alice, alice@example.com, 192.168.1.1`
- If validation fails: prints which field failed and why to stderr
- At the end prints total valid and invalid record counts
- Uses `BASH_REMATCH` to extract the domain part from the email

> 📄 `03_Bash_Scripting.md` → 3.11 Regex in Bash — =~, Character Classes, Anchors

---

S14 — File Descriptor Manipulation & Logging

Write a script that:

- At the very top redirects all stdout **and** stderr for the entire script to both the terminal and `/var/log/myscript.log` simultaneously — using `exec` and process substitution with `tee`
- Opens a custom file descriptor `3` for writing to a separate `/var/log/myscript_errors.log`
- Has a `log_info` function that writes to normal stdout
- Has a `log_error` function that writes to both stderr **and** file descriptor `3`
- Demonstrates all three with sample messages
- Closes file descriptor `3` at the end
- Uses `trap` to ensure cleanup on exit

> 📄 `03_Bash_Scripting.md` → 3.13 File Descriptor Manipulation + 3.9 Exit Codes & Error Handling

---

S15 — Retry with Exponential Backoff

Write a production-grade `retry` function that:

- Is called `retry` and accepts: max attempts, initial delay, and a command to run
- Runs the command — if it succeeds, returns immediately with exit `0`
- If it fails: prints `Attempt N of MAX failed. Retrying in Xs...`
- Doubles the delay after each failure (exponential backoff)
- After all attempts exhausted: prints `All N attempts failed` to stderr and returns exit `1`

Write a main section that uses this function to:

- Retry `curl -sf https://api.example.com/health` up to 5 times
- Retry a `pg_dump` command up to 3 times

Handles the case where the command itself contains spaces and arguments correctly using `"$@"`

> 📄 `03_Bash_Scripting.md` → 3.5 Loops + 3.6 Functions + 3.9 Exit Codes & Error Handling + 3.15 Script Hardening

---

S16 — Complete Deployment Script

Write a production-grade deployment script that combines everything so far:

- Full strict mode, IFS, `readonly` constants
- Accepts arguments: `environment`, `version`, optional `--dry-run` flag
- Has a `parse_args` function that handles both positional and optional arguments
- Has a `log` function with levels: `INFO`, `WARN`, `ERROR` — errors go to stderr
- Has a `cleanup` function trapped on `EXIT`
- Uses a `flock` lockfile to prevent concurrent deployments — prints `Deploy already running` and exits if lock is held
- Validates all inputs upfront before doing anything
- Has a `dry_run` mode where it prints every command prefixed with `[DRY-RUN]` but does not execute it
- At the end prints a summary: `Deployment of vX.Y.Z to production completed in Xs`

> 📄 `03_Bash_Scripting.md` → 3.15 Script Hardening + 3.6 Functions + 3.9 Exit Codes & Error Handling + 3.2 Variables

---

S17 — Log Analyser Script

Write a script that analyses an nginx access log passed as an argument:

- Validates the file exists, is readable, and is non-empty using file tests
- Uses `awk` inside the script to calculate:
  - Total number of requests
  - Count of each HTTP status code
  - Top 5 client IPs by request count
  - Average response size in KB
- Uses `while IFS= read -r` for line-by-line processing where needed
- Formats output as a clean report with headers using `printf`
- Accepts an optional `--since "1 hour ago"` argument and filters log lines by timestamp
- Exits with code `2` if any 5xx errors exceed 5% of total requests — useful for CI/CD health gates

> 📄 `03_Bash_Scripting.md` → 3.5 Loops + 3.8 String Manipulation + 3.4 Conditionals + `04_Text_Processing.md` → 4.5 awk

---

S18 — Config File Manager

Write a script that manages application config files:

- Takes a template file as input containing placeholders like `{{DB_HOST}}`, `{{APP_PORT}}`, `{{ENV}}`
- Takes an environment argument
- Has an associative array per environment with all variable values
- Replaces all `{{PLACEHOLDER}}` patterns in the template using `sed` — but safely, handling values that contain `/` characters (use alternate delimiter)
- Before writing the output, creates a timestamped backup of the existing config if it exists
- Writes the rendered config to `/etc/app/config.yaml`
- Uses `diff` to show what changed between old and new config
- Validates no unreplaced `{{` placeholders remain in the output — exits with error if any found

> 📄 `03_Bash_Scripting.md` → 3.7 Arrays & Associative Arrays + 3.8 String Manipulation + `04_Text_Processing.md` → 4.4 sed

---

S19 — System Health Check Script

Write a script that performs a complete system health check:

- Has individual functions for each check: `check_disk`, `check_memory`, `check_load`, `check_services`
- `check_disk`: alerts if any filesystem is above 80% — uses `df` and `awk`
- `check_memory`: alerts if available memory drops below 500MB — uses `/proc/meminfo`
- `check_load`: alerts if per-CPU load exceeds 1.5 — uses `uptime` and `nproc`

- `check_services`: takes an array of service names, checks each with `systemctl is-active`
- Each check returns exit code `0` for OK, `1` for WARNING, `2` for CRITICAL
- Main function runs all checks, collects results, prints a summary table using `printf`
- Overall exit code is the highest severity found — `0` if all OK, `1` if any warning, `2` if any critical
- Accepts `--json` flag to output results as JSON instead of human-readable

> 📄 `03_Bash_Scripting.md` → 3.6 Functions + 3.15 Script Hardening + 3.4 Conditionals + 3.7 Arrays

---

S20 — Backup Script with Rotation

Write a production backup script that:

- Full strict mode with complete error handling
- Backs up a directory passed as argument to `/backup/YYYYMMDD_HHMMSS_dirname.tar.gz`
- Before backup: checks source exists, destination has enough free space (at least 2× source size)
- Uses `trap` to clean up incomplete backup files if script is interrupted mid-way
- After successful backup: verifies the archive integrity with `tar -tzf`
- Implements backup rotation: keeps only the last 7 daily backups, deletes older ones
- Sends a summary to a log file including: backup size, duration, files backed up count
- Has a `--list` flag to show existing backups with their sizes and dates
- Has a `--restore BACKUP_FILE DEST_DIR` flag to restore a specific backup

> 📄 `03_Bash_Scripting.md` → 3.9 Exit Codes & Error Handling + 3.15 Script Hardening + 3.6 Functions + 3.5 Loops

---

S21 — Process Monitor & Auto-Restart Daemon

Write a script that acts as a simple process supervisor:

- Takes a command to monitor as its argument (e.g. `./myapp --port 8080`)
- Runs the command as a background process
- Monitors it in a loop every 5 seconds using `kill -0`
- If the process dies: logs the exit code, waits 5 seconds, restarts it
- Implements a circuit breaker: if the process crashes more than 5 times in 60 seconds, stops trying and exits with `CRITICAL: service is crash-looping`
- Handles `SIGTERM` and `SIGINT` to the supervisor gracefully — kills the child process cleanly with `SIGTERM` first, then `SIGKILL` after 10 seconds if it hasn't stopped
- Logs every restart with timestamp to `/var/log/supervisor.log`
- Uses `trap` correctly so cleanup always fires

> 📄 `03_Bash_Scripting.md` → 3.9 Exit Codes & Error Handling + 3.6 Functions + 2.4 Signals & kill + 2.2 Job Control

---

S22 — Secret & Environment Validator

Write a script for a CI/CD pipeline that:

- Defines a list of required environment variables: `DB_HOST`, `DB_PASSWORD`, `API_KEY`, `DEPLOY_ENV`
- Defines a list of optional variables with defaults: `LOG_LEVEL=INFO`, `TIMEOUT=30`, `REPLICAS=2`
- Validates every required variable is set and non-empty — collects **all** missing variables and reports them together (not one at a time)
- Validates specific formats: `DEPLOY_ENV` must match `production|staging|dev`, `TIMEOUT` must be a positive integer, `REPLICAS` must be between 1 and 10
- Masks sensitive values in log output — `DB_PASSWORD` and `API_KEY` should print as `DB_PASSWORD=***`
- Uses `export` correctly so child processes inherit validated values
- At the end prints a clean summary of all variables (masking secrets) so engineers can verify the environment before deployment proceeds

> 📄 `03_Bash_Scripting.md` → 3.2 Variables, Quoting Rules, export, env + 3.11 Regex + 3.15 Script Hardening

---

S23 — Parallel Server Provisioning with Semaphore

Write a script that provisions a fleet of servers in parallel but with controlled concurrency:

- Reads server names from a file passed as argument (one per line)
- Provisions each server by running a `provision_server` function that: SSHs in, installs packages, runs health check
- Runs maximum 4 provisioning jobs in parallel at any time — implement a semaphore pattern in pure bash
- Tracks which servers succeeded and which failed in separate arrays
- Shows a live progress indicator: `Provisioning: 4/20 complete, 1 failed`
- After all done: prints a final report with success/failure lists
- The entire script must handle `SIGINT` cleanly — if Ctrl+C pressed, kills all background jobs and prints how many were completed before interruption
- Uses `wait -n` (bash 4.3+) to efficiently wait for any single job to finish

> 📄 `03_Bash_Scripting.md` → 3.12 xargs & Parallel Execution + 3.9 Exit Codes & Error Handling + 3.7 Arrays + 3.14 Subshells vs Child Processes

---

S24 — Log Rotation & Archival Script

Write a production log management script that:

- Scans `/var/log/app/` for log files matching `*.log`
- For files larger than 100MB: rotates them by renaming to `app.log.YYYYMMDD_HHMMSS` and sending `SIGUSR1` to the application to reopen its log file — never deletes open files
- Compresses rotated logs older than 1 day using `gzip` — but only if no process has the file open (use `lsof` to check)
- Archives compressed logs older than 7 days to `/archive/` — checks destination has enough space first
- Deletes archived logs older than 30 days
- Uses file locking to prevent two instances running simultaneously

- Generates a report: files rotated, files compressed, files archived, files deleted, space freed
- All operations in `--dry-run` mode just print what would happen

> 📄 `03_Bash_Scripting.md` → 3.15 Script Hardening + 3.5 Loops + 3.6 Functions + 2.4 Signals + `06_Storage_IO.md` → 6.1 df, du

---

S25 — Complete SRE Incident Response Script

Write a script that automates first-response data collection during an incident:

- Takes a `--pid PID` optional argument for a specific process to investigate
- Collects and saves to `/tmp/incident_TIMESTAMP/`:
  - `vmstat.txt` — 10 samples at 1 second interval
  - `top.txt` — batch mode snapshot
  - `ps.txt` — full process list with all columns
  - `netstat.txt` — all connections with states
  - `disk.txt` — `df` and `du` of top directories
  - If `--pid` given: also collects `/proc/PID/status`, `/proc/PID/fd` count, `/proc/PID/limits`, open files via `lsof`
- Runs all collections in parallel where possible
- Waits for all to complete with a timeout of 30 seconds — kills any that hang
- Packages everything into a single `incident_TIMESTAMP.tar.gz`
- Prints `Incident data collected: /tmp/incident_TIMESTAMP.tar.gz` at the end
- Uses strict mode, proper error handling, cleanup trap throughout

> 📄 `03_Bash_Scripting.md` → 3.15 Script Hardening + 3.12 Parallel Execution + 2.7 /proc/PID Internals + `07_Performance_Observability.md` → 7.1 vmstat